13. Minimax Algorithm

```
import math

def minimax(depth, node_index, is_maximizing, scores, target_depth):
    # Base case: Leaf node reached
    if depth == target_depth:
        return scores[node_index]

    if is_maximizing:
        return max(
            minimax(depth + 1, node_index * 2, False, scores, target_depth),
            minimax(depth + 1, node_index * 2 + 1, False, scores, target_depth)
        )
    else:
        return min(
            minimax(depth + 1, node_index * 2, True, scores, target_depth),
            minimax(depth + 1, node_index * 2 + 1, True, scores, target_depth)
        )

if __name__ == "__main__":
    # Explicit terminal state evaluation values
    leaf_scores = [3, 5, 2, 9, 12, 5, 23, 23]
    tree_depth = int(math.log2(len(leaf_scores)))

    optimal_value = minimax(0, 0, True, leaf_scores, tree_depth)
    print(f"The optimal value calculated by Minimax is: {optimal_value}")
```

main.py Visualize Run ▶

```python
import math

def minimax(depth, node_index, is_maximizing, scores, target_depth):
    # Base case: Leaf node reached
    if depth == target_depth:
        return scores[node_index]

    if is_maximizing:
        return max(
            minimax(depth + 1, node_index * 2, False, scores, target_depth),
            minimax(depth + 1, node_index * 2 + 1, False, scores, target_depth)
        )
    else:
        return min(
            minimax(depth + 1, node_index * 2, True, scores, target_depth),
            minimax(depth + 1, node_index * 2 + 1, True, scores, target_depth)
        )

if __name__ == "__main__":
```

Output

```
The optimal value calculated by Minimax is: 12
```